

Chapter 17

3)

```
int *create_array(int n, int initial_value) {  
    if (n <= 0) {  
        return NULL;  
    }  
  
    int *array = (int *)malloc(n * sizeof(int));  
    if (array == NULL) {  
        return NULL;  
    }  
  
    for (int i = 0; i < n; i++) {  
        array[i] = initial_value;  
    }  
  
    return array;  
}
```

11)

```
int count_occurrences(struct node *list, int n) {  
    int count = 0;  
    while (list != NULL) {  
        if (list->data == n) {  
            count++;  
        }  
        list = list->next;  
    }  
    return count;  
}
```

15)

The code produces no output because it enters an infinite loop. The function f1 increments n indefinitely since no integer value for n satisfies the condition $n^3 - 12 = 0$. As a result, f2(n) never returns 0, causing f1 to loop endlessly, with n continuously increasing without reaching the desired termination condition.

Chapter 18

1.A)

```
static char **lookup(int level);
```

Storage Class: static

Type Qualifiers: None

Type Specifiers: char

Declarators: **lookup(int level)

Initializers: None

1.B)

```
volatile unsigned long io_flags;
```

Storage Class: None

Type Qualifiers: volatile

Type Specifiers: unsigned long

Declarators: io_flags

Initializers: None

1.C)

```
extern char *file_name[MAX_FILES], path[];
```

Storage Class: extern

Type Qualifiers: None

Type Specifiers: char

Declarators: *file_name[MAX_FILES], path[]

Initializers: None

1.D)

```
static const char token_buf[] = "";
```

Storage Class: static

Type Qualifiers: const

Type Specifiers: char

Declarators: token_buf[]

Initializers: "" (an empty string)

3)

```
extern float a;
```

- **Storage Duration:** Static (because extern implies static storage duration)
- **Scope:** File
- **Linkage:** External (can be accessed from other translation units)

```
void f(register double b)
```

- **b** (parameter of f):
 - **Storage Duration:** Automatic (parameters have automatic storage duration by default)
 - **Scope:** Block (local to the function f)
 - **Linkage:** None (parameters do not have linkage)
- **f** (function itself):
 - **Storage Duration:** Static (functions have static storage duration by default)
 - **Scope:** File
 - **Linkage:** External (unless specified otherwise, functions have external linkage)

static int c;

- **Storage Duration:** Static (explicitly specified by static)
- **Scope:** File
- **Linkage:** Internal (limited to this translation unit due to static)

auto

The keyword auto is redundant here because it is the default storage class for local variables within functions. However, if there were a variable like auto int d; within a function:

- **Storage Duration:** Automatic
- **Scope:** Block (local to the function in which it's declared)
- **Linkage:** None (automatic variables do not have linkage)

8.A)

char (*x[10])(int);

- x is an array of 10 pointers to functions.
- Each function takes a single int argument and returns a char.

Type of x: Array of 10 pointers to functions that take an int argument and return a char.

8.B)

int (*x(int))[5];

- x is a function that takes a single int argument.
- x returns a pointer to an array of 5 int elements.

Type of x: Function that takes an int argument and returns a pointer to an array of 5 int.

8.C)

float *(*x(void))(int);

- x is a function that takes no arguments (i.e., void).
- x returns a pointer to a function.
- The function pointed to takes a single int argument and returns a pointer to float.

Type of x: Function that takes no arguments and returns a pointer to a function that takes an int argument and returns a pointer to float.

8.D)

void (*x(int, void (*y)(int)))(int);

- x is a function that takes two parameters:
 - An int
 - A pointer y to a function that takes an int argument and returns void.
- x returns a pointer to a function.
- The function pointed to takes an int argument and returns void.

Type of x: Function that takes an int and a pointer to a function (which takes an int argument and returns void) as parameters, and returns a pointer to a function that takes an int argument and returns void.